Dúvidas da parte 1 do exercício 8

Três conferências para cada função

1º passo

O que a função devolve

Um valor? Chame com SELECT nome(...). Uma tabela? Use SELECT * FROM nome(...).

2º passo

O caso vazio

Pedido sem item ou cliente sem compra devolve 0, e não vazio. Use COALESCE.

3º passo

Os dois testes

Um caso que dá certo e um que dá vazio ou erro, com o resultado previsto em comentário.

Funções em PL/pgSQL

Banco de Dados 2 · Aula 18

SQL: Structured Query Language · PL/pgSQL: Procedural Language/PostgreSQL

NOVO ASSUNTO

Variáveis na função

Na aula passada, a função era uma consulta com nome. Com LANGUAGE plpgsql, ela vira um pequeno programa, como o bloco anônimo, mas com nome:

- Busca valores e guarda em variáveis, com DECLARE e SELECT ... INTO.
- Calcula com esses valores e devolve o resultado com RETURN.

É o bloco anônimo da aula passada, agora com parâmetro e com um valor de volta.

Onde e por que usar variáveis

Quando uma consulta só não basta

Vários passos

Buscar e depois calcular

Busca a UF (Unidade Federativa) do pedido e depois calcula o frete dela.

Reaproveitar

Chamar outra função

Guarda o resultado de uma função e usa no passo seguinte.

Conferir

Ver no meio do caminho

RAISE NOTICE mostra o valor de cada variável.

frete_do_pedido (1 de 2)

OBJETIVO **Qual é o frete de um pedido, só pelo número dele?**

```
1 CREATE OR REPLACE FUNCTION   -- cria ou troca
2   frete_do_pedido(p_pedido integer)   -- o pedido
3 RETURNS numeric   -- devolve o frete
4 LANGUAGE plpgsql STABLE AS $$   -- PL/pgSQL
5 DECLARE   -- as variáveis
6   v_uf char(2); v_valor numeric;   -- UF e valor
7   v_frete numeric;   -- o frete
```

Variável começa com v_, parâmetro com p_. O DECLARE vem antes do BEGIN, como no bloco anônimo.
UF: Unidade Federativa.

frete_do_pedido (2 de 2)

OBJETIVO **Buscar, calcular e devolver**

```
1 BEGIN   -- começa
2   SELECT dest_uf, subtotal - desconto   -- colunas
3   INTO v_uf, v_valor   -- nas variáveis
4   FROM pedido WHERE id = p_pedido;   -- do pedido
5   v_frete := calcular_frete(   -- chama e guarda
6     v_uf, v_valor);   -- a da aula passada
7   RETURN v_frete;   -- devolve e encerra
8 END; $$;   -- fim
```

INTO guarda as colunas nas variáveis, na ordem. := dá um valor à variável.

A variável vazia e cheia

OBJETIVO **O que cada variável guarda antes e depois do SELECT ... INTO?**

```
1 RAISE NOTICE 'Antes: % e %',   -- antes do SELECT
2   v_uf, v_valor;   -- NULL e NULL
3 SELECT dest_uf, subtotal - desconto   -- busca
4 INTO v_uf, v_valor   -- guarda
5 FROM pedido WHERE id = p_pedido;   -- do pedido
6 RAISE NOTICE 'Depois: % e %',   -- depois do SELECT
7   v_uf, v_valor;   -- SP e 180.54
```

Ponha os dois RAISE NOTICE no corpo e chame frete_do_pedido(7). As mensagens aparecem na aba de saída do cliente.

Chamar a função

OBJETIVO **Qual é o frete dos pedidos 7 e 3?**

```sql
SELECT frete_do_pedido(7);   -- preveja: 15.00 (SP)
SELECT frete_do_pedido(3);   -- preveja: 30.00 (RS)
```

Os dois caem no frete da tabela: SP custa 15 e RS custa 30. A função chama a calcular_frete da aula passada.

NOVO ASSUNTO

Decidir com IF

O CASE escolhe um valor dentro de uma consulta. Na função em plpgsql, quem escolhe o caminho é o IF:

- Cada teste é um ELSIF, e o que sobra cai no ELSE.
- Um RAISE EXCEPTION para a função quando algo está errado.

Vamos reescrever a calcular_frete da aula passada com IF e ver a diferença.

Onde e por que usar IF

Três decisões da loja

Faixa

Escolher pelo valor

O frete muda conforme o valor do pedido.

Regra

Recusar o que está errado

A UF (Unidade Federativa) que a loja não atende para a função com uma mensagem.

Motivo

Dizer por quê

O cupom não vale: vencido, esgotado ou abaixo do mínimo.

Do CASE ao IF

A mesma regra, escrita de dois jeitos

Aula passada · CASE

WHEN ... THEN ...

Cada WHEN é um teste. O primeiro verdadeiro escolhe o valor. ELSE: o que sobrou.

Hoje · IF

IF ... ELSIF ... ELSE

Cada ELSIF é um teste. O primeiro verdadeiro roda o seu RETURN. ELSE: o que sobrou.

Um só END IF fecha o IF inteiro, com todos os ELSIF.

calcular_frete em PL/pgSQL (1 de 3)

OBJETIVO **Mesmo nome e mesmos parâmetros da aula passada**

```
1 CREATE OR REPLACE FUNCTION   -- cria ou troca
2   calcular_frete(p_uf char,   -- a UF
3     p_valor numeric)   -- o valor
4 RETURNS numeric   -- o mesmo retorno
5 LANGUAGE plpgsql STABLE AS $$   -- PL/pgSQL
6 DECLARE v_frete numeric;   -- frete da tabela
7 BEGIN   -- começa o corpo
```

O CREATE OR REPLACE troca a função da aula passada: quem já a usa passa a usar a nova. UF: Unidade Federativa.

calcular_frete em PL/pgSQL (2 de 3)

OBJETIVO **Buscar o frete e parar se a loja não atende a UF**

```
1 SELECT valor INTO v_frete   -- busca o frete
2 FROM frete_uf WHERE uf = p_uf;   -- da UF
3 IF NOT FOUND THEN   -- não achou a UF
4   RAISE EXCEPTION   -- para a função
5     'A loja não entrega na UF %',   -- motivo
6     p_uf;   -- o % é a UF
7 END IF;   -- fim do teste
```

IF NOT FOUND confere o SELECT ... INTO logo acima. Na aula passada, a UF AM devolvia vazio; agora para com o motivo.

De onde vem o FOUND?

Uma variável que o PL/pgSQL cria sozinho

Quem cria

O próprio PL/pgSQL

FOUND já existe em toda função plpgsql. Você não declara. Ela começa false a cada chamada.

Quem muda

O último comando

SELECT ... INTO achou uma linha: true. Não achou: false. UPDATE, INSERT e DELETE também mudam.

Como usar

Logo depois do comando

Teste IF NOT FOUND na linha seguinte: o próximo comando troca o valor.

Só existe em PL/pgSQL (Procedural Language/PostgreSQL), não em LANGUAGE sql.

O FOUND na prática

OBJETIVO **O que o FOUND vale depois de cada SELECT ... INTO?**

```
1 DO $$ DECLARE v numeric; BEGIN   -- um bloco de teste
2   SELECT valor INTO v FROM frete_uf   -- busca
3   WHERE uf = 'SP';   -- SP existe
4   RAISE NOTICE 'SP: %', FOUND;   -- t
5   SELECT valor INTO v FROM frete_uf   -- busca
6   WHERE uf = 'AM';   -- AM não existe
7   RAISE NOTICE 'AM: %', FOUND;   -- f
8 END $$;   -- fim
```

Preveja e rode: SP: t e AM: f. No AM, a variável v fica vazia.

calcular_frete em PL/pgSQL (3 de 3)

OBJETIVO **Decidir o frete com IF … ELSIF … ELSE**

```
1    IF p_valor ⩾ 300 THEN RETURN 0;   -- grátis
2    ELSIF p_valor ⩾ 200 THEN RETURN 10;   -- até 299
3    ELSIF p_uf IN ('BA', 'PE') THEN   -- Nordeste
4      RETURN 30;   -- 30 reais
5    ELSIF p_valor < 50 THEN RETURN 40;   -- pequeno
6    ELSE RETURN v_frete;   -- o da tabela
7    END IF;   -- um END IF
8  END; $$;   -- fim
```

Cada WHEN do CASE virou um ELSIF. O RETURN devolve e encerra a função.

Prever o caminho de cada chamada

OBJETIVO **Em que teste cada chamada para?**

```
1 SELECT calcular_frete('SP', 150);   -- 15.00: ELSE
2 SELECT calcular_frete('SP', 250);   -- 10: 2° teste
3 SELECT calcular_frete('BA', 80);   -- 30: 3° teste
4 SELECT frete_do_pedido(7);   -- 15.00: usa a nova
5 SELECT calcular_frete('AM', 80);   -- erro
```

A frete_do_pedido não mudou e já usa a versão nova: a regra está num lugar só. A última linha para com: A loja não entrega na UF AM.

NOVO ASSUNTO

Linha inteira e vários motivos

Às vezes a função precisa de várias colunas da mesma linha, e de vários testes em sequência:

- Uma variável %ROWTYPE guarda a linha inteira, e você lê c.coluna.
- Cada IF com RETURN encerra a função no primeiro motivo encontrado.

É assim que a loja diz por que um cupom não pode ser usado.

situacao_do_cupom (1 de 4)

OBJETIVO **Este cupom pode ser usado numa compra deste valor?**

```
1 CREATE OR REPLACE FUNCTION   -- cria ou troca
2   situacao_do_cupom(p_codigo varchar,   -- o código
3     p_valor numeric)   -- o valor da compra
4 RETURNS text   -- 'ok' ou o motivo
5 LANGUAGE plpgsql STABLE AS $$   -- PL/pgSQL
6 DECLARE c cupom%ROWTYPE;   -- a linha do cupom
7   v_usos integer;   -- quantos usos
```

cupom%ROWTYPE: uma variável com todas as colunas da tabela cupom. Você lê cada coluna com ponto: c.codigo, c.valido_ate.

situacao_do_cupom (2 de 4)

OBJETIVO **Buscar a linha inteira e tratar o cupom que não existe**

```
1 BEGIN   -- começa o corpo
2   SELECT * INTO c FROM cupom   -- a linha toda vai para c
3   WHERE codigo = p_codigo;   -- do código recebido
4   IF NOT FOUND THEN   -- não achou
5     RAISE EXCEPTION   -- para
6       'Cupom % não existe', p_codigo;   -- o motivo
7   END IF;   -- fim do teste
```

SELECT * com INTO c: todas as colunas do cupom vão para a variável c de uma vez.

situacao_do_cupom (3 de 4)

OBJETIVO **O primeiro motivo encerra a função**

```
1 IF c.valido_ate < CURRENT_DATE THEN   -- vencido?
2   RETURN 'Cupom vencido';   -- encerra aqui
3 END IF;   -- o resto não roda
4 SELECT COUNT(*) INTO v_usos   -- conta
5 FROM pedido WHERE cupom_id = c.id   -- os usos
6   AND status NOT IN ('carrinho',   -- não contam
7     'cancelado');   -- nem este
```

Cupom vencido nem chega a contar os usos: o RETURN encerra a função. É o mesmo padrão das funções 6 e 9 do exercício.

situacao_do_cupom (4 de 4)

OBJETIVO **Os outros motivos e o ok**

```
1 IF v_usos ⩾ c.limite_usos THEN   -- esgotado?
2   RETURN 'Cupom esgotado';   -- encerra aqui
3 END IF;   -- fim do teste
4 IF p_valor < c.valor_minimo THEN   -- pequena?
5   RETURN 'Compra abaixo do mínimo de '   -- texto
6     || c.valor_minimo;   -- e o valor
7 END IF;   -- fim do teste
8 RETURN 'ok'; END; $$;   -- passou por tudo
```

|| junta o texto e o número numa frase só.

Um cupom de cada motivo

OBJETIVO **O que a loja responde para cada cupom?**

```sql
SELECT codigo,   -- o cupom
  situacao_do_cupom(codigo, 100)   -- a resposta
FROM cupom   -- dos cupons
WHERE codigo IN ('BEMVINDO10',   -- ok
  'VENCIDO15', 'ESGOTADO5',   -- vencido, esgotado
  'MINIMO500');   -- mínimo
SELECT LEAST(50, 30), LEAST(20, 30);   -- 30 e 20
```

Antes de rodar, preveja a resposta de cada cupom. LEAST devolve o menor dos valores: ele aparece na função 6 do exercício.

NOVO ASSUNTO

Ler uma função em voz alta

Ler o código como uma frase em português ajuda a entender e a achar o erro:

- «Se não existe pedido com este id, pare e diga o motivo.»
- «Se o status é entregue, devolva a frase com a data.»

Vamos ler juntos a função que responde ao cliente que pergunta: «cadê o meu pedido?».

Dois jeitos de testar se a linha existe

Os dois servem para o «não existe»

Depois de buscar

IF NOT FOUND

Logo depois de um SELECT ... INTO. Use quando você precisa dos valores da linha.

Antes de buscar

IF NOT EXISTS (SELECT 1 ...)

Pergunta se existe, sem guardar nada. Use quando só quer saber se existe.

A função 10 do exercício usa IF NOT EXISTS para o cliente que não existe.

situacao_do_pedido (1 de 3)

OBJETIVO **O pedido existe?**

```
1 IF NOT EXISTS (SELECT 1 FROM pedido   -- existe?
2     WHERE id = p_pedido) THEN   -- este pedido
3   RAISE EXCEPTION   -- não existe: para
4     'Pedido % não existe', p_pedido;   -- motivo
5 END IF;   -- fim do teste
```

Leia em voz alta: «se não existe pedido com este id, pare e diga o motivo». O cabeçalho e o DECLARE estão no arquivo da demonstração.

situacao_do_pedido (2 de 3)

OBJETIVO **Busque o status e as datas**

```
1 SELECT status, enviado_em,   -- colunas
2   entregue_em   -- e a entrega
3 INTO v_status, v_enviado,   -- variáveis
4   v_entregue   -- na mesma ordem
5 FROM pedido WHERE id = p_pedido;   -- do pedido
```

Leia em voz alta: «busque o status e as datas do pedido e guarde nas três variáveis».

situacao_do_pedido (3 de 3)

OBJETIVO **Uma frase para cada status**

```
1 IF v_status = 'carrinho' THEN   -- no carrinho
2   RETURN 'Carrinho aberto';   -- a frase
3 ELSIF v_status = 'entregue' THEN   -- chegou
4   RETURN 'Entregue em ' ||   -- texto e
5     TO_CHAR(v_entregue, 'DD/MM/YYYY');   -- data
6 ELSE RETURN 'Outra situação';   -- o resto
7 END IF;   -- fim
```

Aqui aparecem 2 dos 6 status. No arquivo da demonstração estão os 6, e o ELSE fica só com o cancelado.

A função dentro de uma consulta

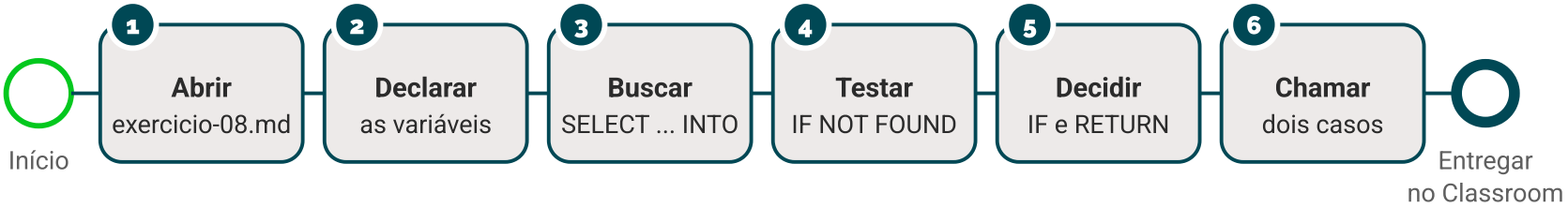
OBJETIVO **O que a loja responde para cada pedido?**

```sql
SELECT status,   -- o status gravado
  situacao_do_pedido(id) AS resposta   -- a frase
FROM pedido   -- dos pedidos
WHERE id ⩽ 12   -- os 12 primeiros
ORDER BY id;   -- em ordem
```

12 linhas, com os 6 status. Tente também situacao_do_pedido(999999): Pedido 999999 não existe.

Exercício 8, parte 2: funções em PL/pgSQL

Pode fazer em dupla. Cada aluno entrega o seu, no Classroom



> Faça as funções 5 a 10, uma de cada vez. Antes de chamar, escreva em um comentário o que você espera.

Para fixar: responda de memória

Sem olhar o material e sem nota. Escreva as respostas em comentários, no fim do arquivo

Pergunta 1

DECLARE

Para que serve o DECLARE?

Pergunta 2

RAISE EXCEPTION

O que o RAISE EXCEPTION faz com a função?

Pergunta 3

SELECT ... INTO

Onde o SELECT ... INTO guarda o resultado?